03. MR数据处理流程类面试题

由 涤生老师树哥创建, 最后修改于三月 16, 2025

- 1.MapReduce的Shuffle过程？(☆☆☆)
  - 1.1 shuffle小趴菜版
  - 1.2shuffle精简版本
  - 1.3 shuffle炫酷版
  - 1.4 MR编程模式实战图解
- 2. 为什么需要shuffle?什么场景需要shuffle?(☆☆☆)
- 3. MR处理文件的切片大小配置？InputFormat类？(☆☆☆)
  - 3.1 split切片大小?
  - 3.2 文件切片机制?
- 4. Maptask的个数由什么决定？(☆☆☆)
- 5.为什么在Map和Reduce阶段系统默认必须要对Key进行排序？(☆☆☆)
- 6.shuffle过程有几次排序，几次落盘？(☆☆☆)
- 7.请说下MR中Map Task的工作机制(☆☆☆)
- 8. 请说下MR中Reduce Task的工作机制(☆☆☆)
- 9.Word Count的图要能看得懂!
- 10.MapReduce从提交到最后执行大概是一个什么过程
- 11.Mapreduce中combiner发生在什么时候，什么时候使用?
- 12.小文件的产生和治理
- 13.MR优化的方式
- 14.MR跑的慢的原因：减少落盘次数，那不落盘的话，数据放在哪里，是内存吗?
- 15.没有排序需求就可以使用hashshuffle吗?
- 16.spill过多，怎么优化
- 17.数据倾斜可能是什么原因，怎么发现的，如何处理
- 18.Shulffle过程瓶颈在哪里，你会怎么解决?

- ☆ **表示了解即可，要知道个大概，增加知识面，不要求必须掌握**
- ☆☆ **表示尽可能背熟，加上一些自己的理解更好;**
- ☆☆☆ **表示需要理解，必须掌握，最好能结合自己话说表达出来;**

1.MapReduce的Shuffle过程？(☆☆☆)

尖叫提示：整个Hadoop课程中，概率最高的题目，看下面的统计就懂了★★★

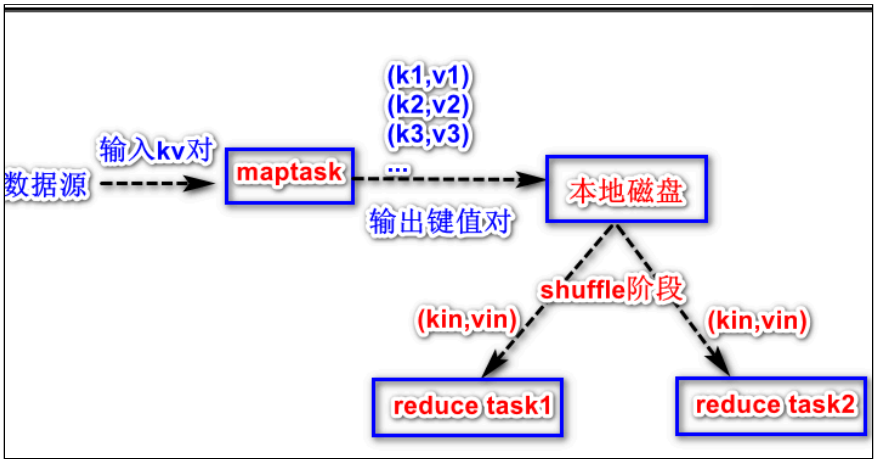
问过的一些公司：【7.阿里淘宝天猫社招面试题（2. 2024年5月4年同学面试），10.美团社招大数据面试（3.美团到店2023，4. 美团优选高级数据研发，5.美团优选高级数据研发)】

课程链接：5.4.6.Shuffle与排序☆☆☆

衍生问题： MR流程，shuffle过程

参考答案：

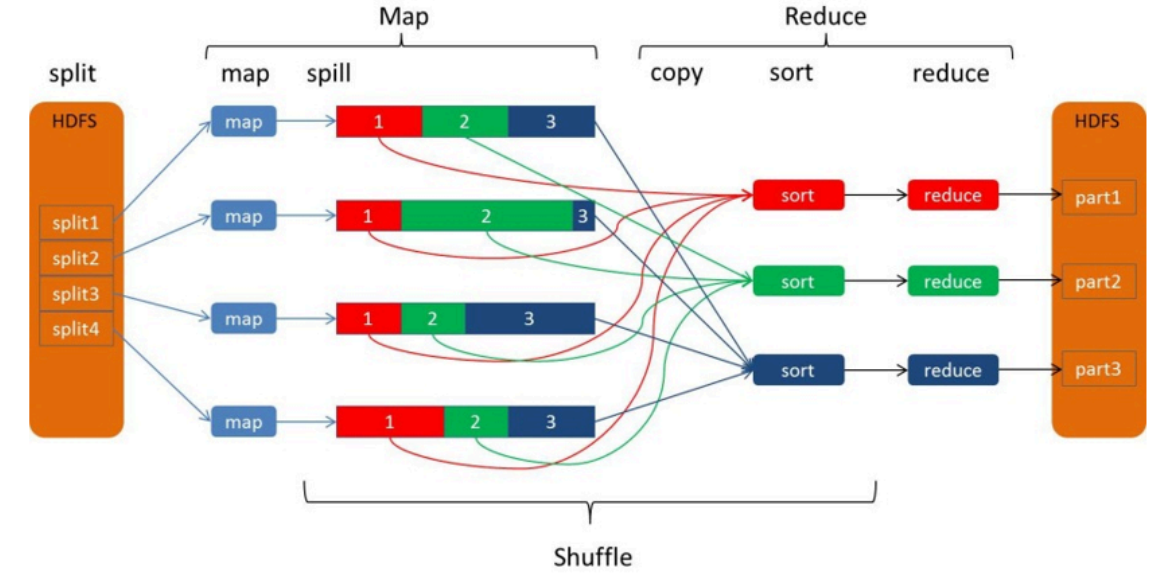
1.1 shuffle小趴菜版



1. **maptask将文本按行切分形成kv键值对，键值对先放在内存中进行排序分区，然后落到本地磁盘;**
2. **reducetask启动服务去每个map上拉取属于的自己的数据，然后将多个map上拉取的数据进行合并排序，最后将这个结果进行进一步操作，shuffle结束。**

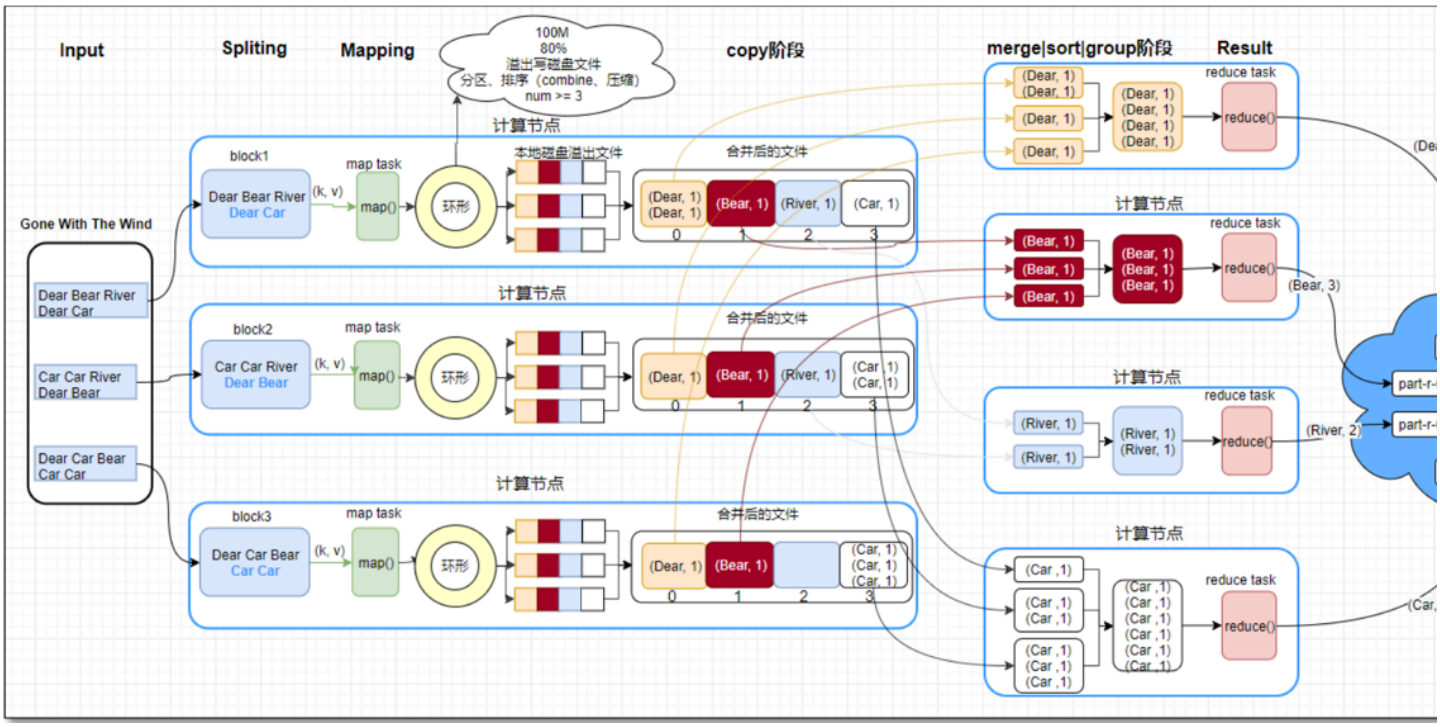
1.2shuffle精简版本

Map方法之后，Reduce方法之前的数据处理过程称之为Shuffle。所谓的shuffle正如其字面意思洗牌，就是将map的结果二次分发分布到reduce的过程。



1. maptask 收集我们的 map() 方法输出的 kv 对，放到内存缓冲区中
2. 从内存缓冲区不断溢出本地磁盘文件（非HDFS,因为是中间结果没必要），可能会溢出多个文件
3. 多个溢出文件会被合并成大的溢出文件
4. 在溢出过程中，及合并的过程中，都要调用 partitoner 进行分组和针对 key 进行排序
5. reducetask 根据自己的分区号，去各个 maptask 机器上取相应的结果分区数据
6. reducetask 会取到同一个分区的来自不同 maptask 的结果文件， reducetask 会将这些文件再进行合并（归并排序）
7. 合并成大文件后， shuffle 的过程也就结束了，后面进入 reducetask 的逻辑运算过程（ 从文件中取出一个一个的键值对 group ，调用用户自定义的 reduce() 方

1.3 shuffle炫酷版



Map方法之后，Reduce方法之前的数据处理过程称之为Shuffle。所谓的shuffle正如其字面意思洗牌，就是将map的结果二次分发分布到reduce的过程。

整体流程：collect阶段---> Map阶段---> Collect阶段---> spill阶段----> Combine阶段

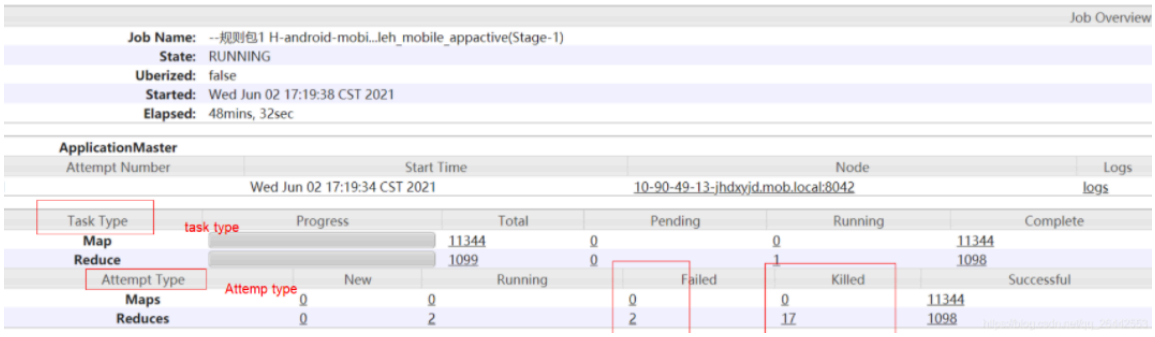
1. 准备前奏：客户端首先获取待处理文件信息，然后根据集群与客户端参数配置，形成一个任务分配的规划。再调用submit()方法，把要执行的jar包、Job.xml(Job信息)、Job.split（分片信息）等提交到hdfs集群。然后把程序提交到yarn集群运行，yarn会生成一个MrAppMaster（一个进程），用来控制maptask和reducetask的启动。因为分片信息已经提交到hdfs集群，那MrAppMaster就会去获取分片信息，计算出Maptask数量。MapTask通过用户编写的RecordReader，从输入InputSplit中解析出一个个key/value。
2. maptask主将解析出的key/value交给map()函数处理，并产生一系列新的key/value。maptask 收集我们的 map() 方法输出的 kv 对，并写入到环形内存缓冲区中
3. 当环形缓冲区满80%后，就会打开流，MapReduce会将数据写到本地文件系统磁盘上，生成一个临时文件，然后关闭流。当环形缓冲区再次满80%后，又会打开流，开始溢写。因此，有可能生成多个临时文件。需要注意的是，将数据写入本地磁盘之前，先要对数据进行一次本地*排序*，并在必要时对数据进行规约、压缩、分区等操作
4. 当所有数据处理完成后，MapTask对所有临时文件进行一次合并，以确保最终只会生成一个数据文件。在进行文件合并过程中，MapTask以分区为单位进行合并，对于某个分区，它将采用多轮递归合并的方式。在溢出过程中，及合并的过程中，都要调用 partitoner 进行分组和针对 key 进行归并*排序*
5. reduce两步走（copy数据-->merge+sort）：Reduce任务通过HTTP向各个Map任务拖取它所需要的数据。每个节点都会启动一个常驻的HTTP server，其中一项服务就是响应Reduce拖取Map数据。 具体来说就是reducetask 根据自己的分区号，去各个 maptask 机器上取相应的结果分区数据。注意如果在内存中能放得下这次数据的话就直接把数据写到内存中，Reduce要向每个Map去拖取数据，在内存中每个Map对应一块数据，当内存中存储的Map数据占用空间达到一定程度的时候，开始启动内存中merge，把内存中的数据merge输出到磁盘上一个文件中。如果在内存中不能放得下这个Map的数据的话，直接把Map数据写到磁盘上，在本地目录创建一个文件，从HTTP流中读取数据然后写到磁盘。一般Reduce是一边copy一边sort*归并排序*，即copy和sort两个阶段是重叠而不是完全分开的。
6. 如果形成了多个磁盘文件还会进行合并，最后一次合并的结果作为reduce的输入而不是写入到磁盘中。合并成大文件后， shuffle 的过程也就结束了，后面进入 reducetask 的逻辑运算过程；

尖叫总结

1. .map任务将输出的结果时写到本地磁盘，并不是HDFS系统，因为map输出的是中间结果，等reduce处理完以后，会从本地磁盘删除，所以没有必要上传到hdfs上，还会形成复本。这也是MapReduce比Spark慢的原因之一，因为Spark是将中间结果缓存到内存里，再次使用的时候直接加载，比mapreduce还有从本地磁盘读取快了太多。

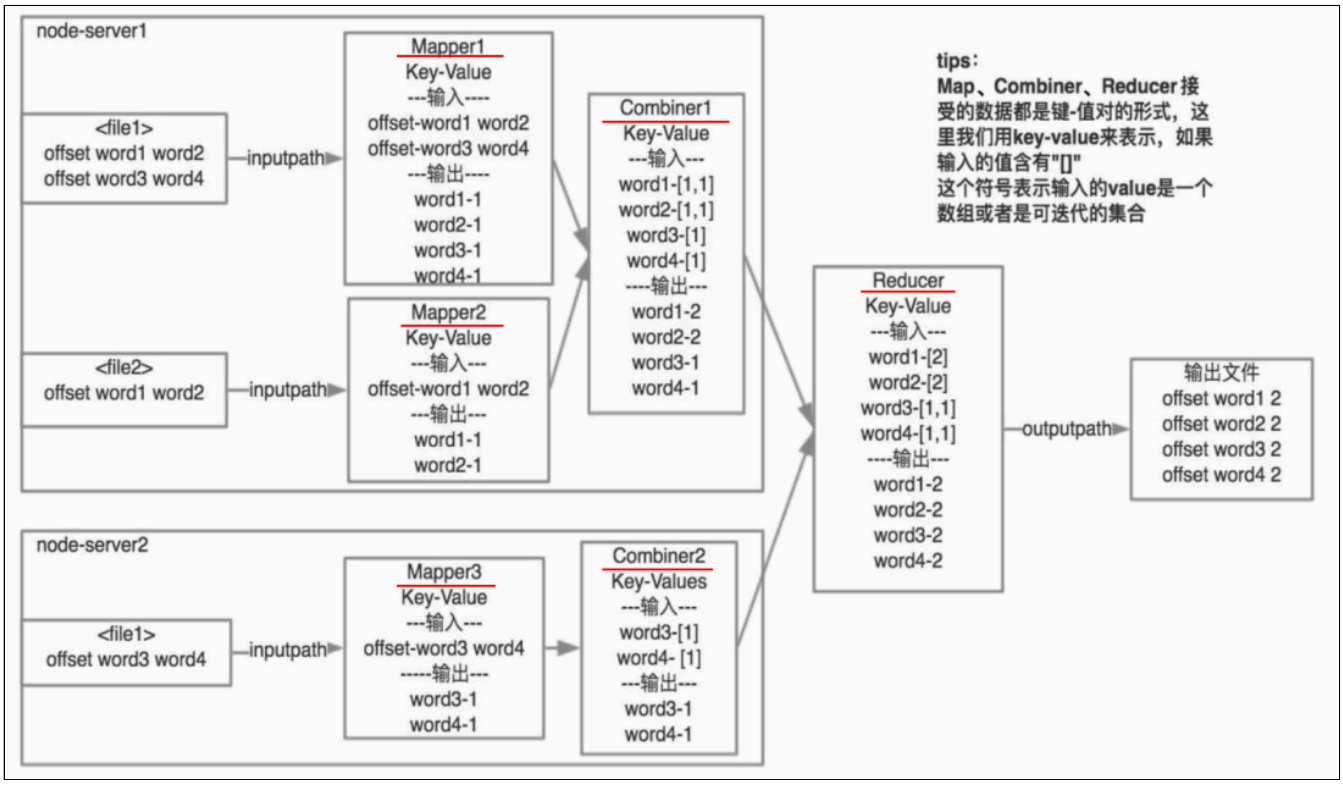
2. **Shuffle 中的缓冲区大小会影响到 mapreduce 程序的执行效率，原则上说，缓冲区越大，磁盘 io 的次数越少，执行速度就越快。缓冲区的大小可以通过参数调整，参数：io.sort.mb 默认 100M。生产集群一般256Mb起，一般都是512MB,1024Mb的配置。但是注意一点，不管数据多大，都是要落盘的。**
3. **如果运行map任务的节点在将map输出的结果传送给reduce之前就失败了。那么Hadoop将在另一个节点上重新运行这个map任务以再次构建新的map中间结果。MapReduce框架好就好在程序员不用担心系统失效的问题，因为框架可以检测到失败的任务，并重新在正常的机器上启动执行，这个主要是由MRappmaster进行监控的任务情况。如果失败让yarn进行调度分配新的container.**

4. **生产上每个map/reduce的实例默认可以失败重试4次，一般在某个节点上失败后，就不会再给这个节点执行了，而是分配新的节点执行这个maptask/reducetask 。如果超过四次直接整个任务就挂了。当然这个参数可以配置更大，一般不修改**



5. **MapTask和ReduceTask均会对数据按照key进行排序。该操作属于Hadoop的默认行为。任何应用程序中的数据均会被排序，而不管逻辑上是否需要否需要。默认排序是按照字典顺序排序，且实现该排序的方法是快速排序。**

1.4 MR编程模式实战图解



2. 为什么需要shuffle?什么场景需要shuffle?(☆☆☆)

衍生问题：谈谈你对shuffle的理解，shuffle的做用是什么？哪些场景会出现shuffle?

什么是shuffle,谈谈你对其的理解?

1. **shuffle的目的是将map端处理的结果二次分发分布给redcue端进行进一步处理。比如将所有的map的结果集按照key进行hash散列到不同的reduce上处理，保障使得同样的key进入相同的reducer进行聚合处理。**
2. **shuffle移动了大量的数据，对计算、内存、网络和磁盘都有巨大的消耗，因此，只有确实需要shuffle的地方才应该进行shuffle，否则尽可能避免shuffle。这也是为什么有些任务能只在map端完成就在map端完成，因为此时不需要Shuffle，效率会高。**

3. MR处理文件的切片大小配置？InputFormat类？(☆☆☆)

问答总结：

3.1 split切片大小?

客户端首先获取待处理文件信息，然后根据集群与客户端参数配置，形成一个任务分配的规划。再调用submit()方法，把要执行的jar包、Job.xml(Job信息)、Job.split（分片信息）等提交到hdfs集群。然后把程序提交到yarn集群运行，yarn会生成一个MrAppMaster（一个进程），用来控制maptask和reducetask的启动。因为分片信息已经提交到hdfs集群，那

MrAppMaster就会去获取分片信息，计算出Maptask数量

（整个切片过程在getSplit()方法中完成）
默认情况下，切片大小等于块block大小(128M)，对每一个文件进行切片，而不是整体的数据。并且每次切片时判断剩余文件部分是否大于切片大小的1.1倍，若小于切片大小，则剩余部分形成一个切片，大于则继续按切片大小切片，再继续切分。切片完成后，切片信息提交到yarn(mrAppMaster)，mrAppMaster会启动相对应数量的MapTask，也就是有多少个切片就有多少个MapTask。

思考一下：为什么什么设置切片大小等于块大小?

因为如果切片不设置为和块大小一样大，有的块可能需要跨网络传输拼接切片。本质是为了提高mr计算执行的本地化，既然文件存在哪在哪计算执行。减少集群的网路io和带宽的消耗。

举个例子，一个文件100万行，加入10万行正好一个block（128MB），则有10个block，10个block加入存放在10台机器上；如果split切片不等于block大小，比如切片是130MB,那么第一个切片就需要跨两个节点进行网络传输数据才能完成切片。

思考一下：所有文件都支持切片吗?

并不是所有的文件都支持切片。有一些压缩文件，使用的压缩算法不支持切片，最终就只能是一片进行处理。

3.2 文件切片机制?

1. TextInputFormat（系统默认）:按行读取每条记录。key：该行在整个文件中的起始字节偏移量，LongWritable类型。value：这行的内容，Text类型。切片机制：对任务按文件规划切片，不管文件多小，都会是一个单独的切片，都会交给一个MapTask，这样如果有大量小文件，就会产生大量的MapTask，处理效率极其低下。
2. CombineTextInputFormat：用于小文件过多的场景，它可以将多个小文件从逻辑上规划到一个切片中，这样，多个小文件就可以尽可能的交给一个MapTask处理。

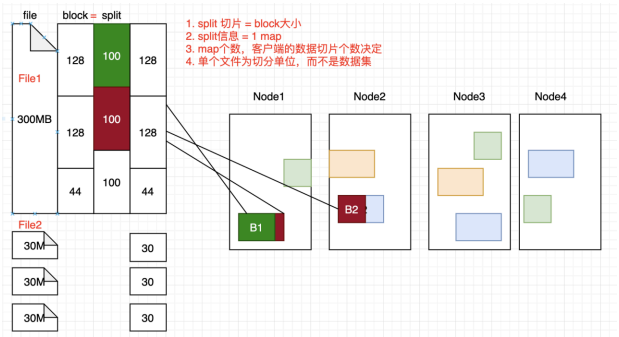
4. Maptask的个数由什么决定? (☆☆☆)

问过的一些公司：【1.字节跳动社招面试题（2.字节跳动）】

课程链接：5.4.3.数据分片（Split）☆☆，5.4.8.数据输入输出☆

问答总结：

Map的数量主要有输入的文件切片个数决定，而文件的切片个数主要牵涉到文件是否压缩支持切片，每个切片的大小，文件是否开启了map前合并等因素决定。



FileInputFormat切片机制☆

- 简单按照文件的内容长度进行切片
- 切片大小，默认等于Block大小

切片时不考虑数据集整体，而是逐个针对每个文件进行单独的切片

TextInputFormate切片机制☆

MapReduce默认使用的切片机制。在切片时不考虑数据集整体，也就是不考虑输入路径下其他文件，而是逐个对每一个文件单独切片。

它在读取单个文件数据时是按行读取的，其用来存储读取的数据的数据结构固然是键值对，键存储的是该行在整个文件中的起始字节的偏移量，键的数据类型是LongWritable。而值是这行的内容，不包括任何的行终止符，即换行符和回车符，值的数据类型是Text。

切片过程

- 程序找到数据存储的目录
- 开始遍历处理目录下的每个文件
- 遍历到第一个文件 f.txt
- 获取文件大小 fs.size(f.txt)
- 计算切片大小 computeSplitSize(Math.max(minSize, Math.min(maxSize,blocksize)))=blocksize=256M
- 默认切片大小=blocksize
- 开始切片, 0-256M, 256-512M (每次切片时,都要判断切完剩余部分是否大于block 的 1.1 倍, 不大于就划分为 1 块)
- 建切片信息记录到规划文件中(记录每个切片元数据:起止位置)
- 提交切片规划文件到 YARN 上, YARN 上的 MrAppMaster根据规划文件计算 mapTask 调度资源执行

5.为什么在Map和Reduce阶段系统默认必须要对Key进行排序? (☆☆☆)

问答总结：

因为在Reduce阶段是将相同Key的文件进行合并，如果不对key进行排序，需要一个一个判断是否是相同的key，加大reduce端排序的压力。效率不高；

而按key排好序之后，只需要与后面的相比看是否是相同的key，如果不一样，直接将前面相同key的一组拿走进入到reduce，效率高。

总之，排序后的数据对于后续的使用方便很多。

6.shuffle过程有几次排序，几次落盘？(☆☆☆)

问答总结：

对于 MapTask：

溢写：它会将处理的结果暂时放到环形缓冲区中， 当环形缓冲区使 用率达到一定阈值后，在溢写之前，对缓冲区中的数据进行一次快速排序，之后将这些有序数据溢写到磁盘上，

归并：而当数据处理完毕后，它会对磁盘上所有文件进行**归并排序**。

对于 ReduceTask ，

它从每个 MapTask 上远程拷贝相应的数据文件，先溢写到内存，如果文件大 小超过一定阈值，则溢写磁盘上，否则存储在内存中。

如果磁盘上文件数目达到 一定阈值，则进行一次 归并排序 以生成一个更大文件；如果内存中文件大小或者 数目超过一定阈值，则进行一次合并后将数据溢写到磁盘上。

当所有数据拷贝完 毕后ReduceTask统一对内存和磁盘上的所有数据进行一次归并排序。

7.请说下MR中Map Task的工作机制(☆☆☆)

参考答案：

1) 读取数据组件 InputFormat (默认 TextInputFormat) 会通过 getSplits 方法对输入目录中的文件进行逻辑切片规划得到 block，有多少个 block就对应启动多少个 MapTask。

2) 将输入文件切分为 block 之后，由 RecordReader 对象 (默认是LineRecordReader) 进行读取，以 \n 作为分隔符, 读取一行数据, 返回 \<key，value\>， Key 表示每行首字符偏移值，Value 表示这一行文本内容。

3) 读取 block 返回 \<key,value\>, 进入用户自己继承的 Mapper 类中，执行用户重写的 map 函数，RecordReader 读取一行这里调用一次。

4) Mapper 逻辑结束之后，将 Mapper 的每条结果通过 context.write 进行collect数据收集。在 collect 中，会先对其进行分区处理，默认使用 HashPartitioner。

5) 接下来，会将数据写入内存，内存中这片区域叫做环形缓冲区(默认100M)，缓冲区的作用是 批量收集 Mapper 结果，减少磁盘 IO 的影响。我们的 Key/Value 对以及 Partition 的结果都会被写入缓冲区。当然，写入之前，Key 与 Value 值都会被序列化成字节数组。

6) 当环形缓冲区的数据达到溢写比列(默认0.8)，也就是80M时，溢写线程启动，需要对这 80MB 空间内的 Key 做排序 (Sort)。排序是 MapReduce 模型默认的行为，这里的排序也是对序列化的字节做的排序。

7) 合并溢写文件，每次溢写会在磁盘上生成一个临时文件 (写之前判断是否有 Combiner)，如果 Mapper 的输出结果真的很大，有多次这样的溢写发生，磁盘上相应的就会有多个临时文件存在。当整个数据处理结束之后开始对磁盘中的临时文件进行 Merge 合并，因为最终的文件只有一个写入磁盘，并且为这个文件提供了一个索引文件，以记录每个reduce对应数据的偏移量。

8. 请说下MR中Reduce Task的工作机制(☆☆☆)

问答总结：

Reduce 大致分为 copy、sort、reduce 三个阶段，重点在前两个阶段。

copy 阶段包含一个 eventFetcher 来获取已完成的 map 列表，由 Fetcher 线程去 copy 数据，在此过程中会启动两个 merge 线程，分别为 inMemoryMerger 和 onDiskMerger，分别将内存中的数据 merge 到磁盘和将磁盘中的数据进行 merge。待数据 copy 完成之后，copy 阶段就完成了。

开始进行 sort 阶段，sort 阶段主要是执行 finalMerge 操作，纯粹的 sort 阶段，完成之后就是 reduce 阶段，调用用户定义的 reduce 函数进行处理。

详细步骤：
1) Copy阶段：简单地拉取数据。Reduce进程启动一些数据copy线程(Fetcher)，通过HTTP方式请求maptask获取属于自己的文件（map task 的分区会标识每个map task属于哪个reduce task ，默认reduce task的标识从0开始）。

2) Merge阶段：在远程拷贝数据的同时，ReduceTask启动了两个后台线程对内存和磁盘上的文件进行合并，以防止内存使用过多或磁盘上文件过多。

merge有三种形式：内存到内存；内存到磁盘；磁盘到磁盘。默认情况下第一种形式不启用。当内存中的数据量到达一定阈值，就直接启动内存到磁盘的merge。与map端类似，这也是溢写的过程，这个过程中如果你设置有Combiner，也是会启用的，然后在磁盘中生成了众多的溢写文件。内存到磁盘的merge方式一直在运行，直到没有map端的数据时才结束，然后启动第三种磁盘到磁盘的merge方式生成最终的文件。

3) 合并排序：把分散的数据合并成一个大的数据后，还会再对合并后的数据排序。

4) 对排序后的键值对调用reduce方法：键相等的键值对调用一次reduce方法，每次调用会产生零个或者多个键值对，最后把这些输出的键值对写入到HDFS文件中。

9.Word Count的图要能看得懂!

问过的一些公司：1.字节跳动社招面试题

课程链接：9.5.3.MapReduce编程规范与WordCount案例 - 历史归档

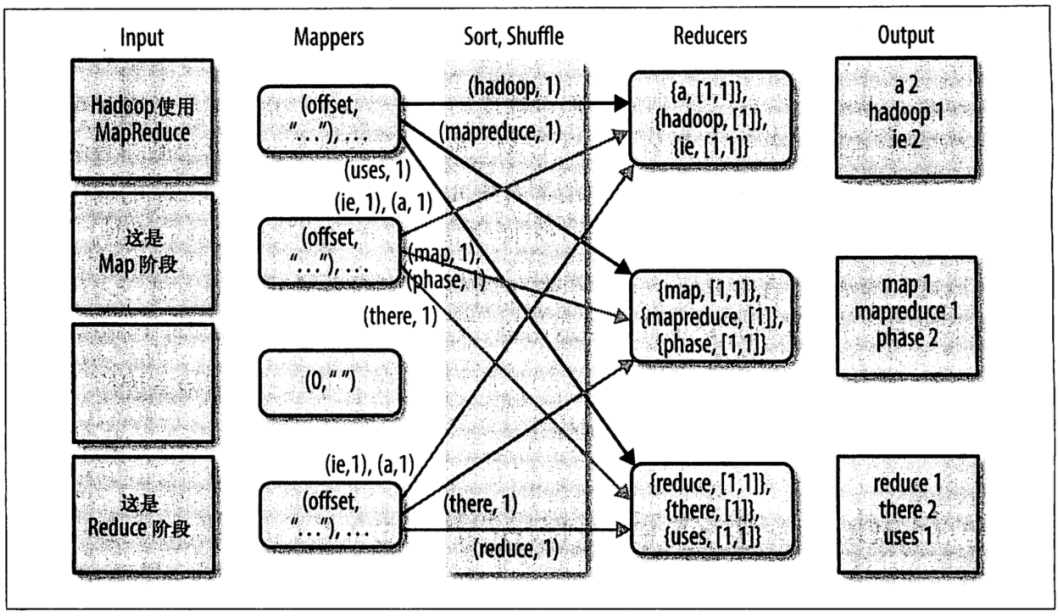
wordcount图是mr的helloworld，要求必须掌握；

Map阶段

1. 设置 InputFormat 类, 将数据切分为 Key-Value(K1和V1)对, 输入到第二步。
2. 自定义 Map 逻辑, 将第一步的结果转换成另外的 Key-Value（K2和V2） 对, 输出结果。

Reduce阶段

1. 对多个 Map 任务的结果进行排序以及合并, 编写 Reduce 函数实现自己的逻辑, 对输入的Key-Value 进行处理, 转为新的 Key-Value（K3和V3）输出。
2. 设置 OutputFormat 处理并保存 Reduce 输出的 Key-Value 数据



10.MapReduce从提交到最后执行大概是一个什么过程

课程链接：9.5.2.Mapreduce的核心思想与数据流流程☆☆

衍生问题： wordcount 流程

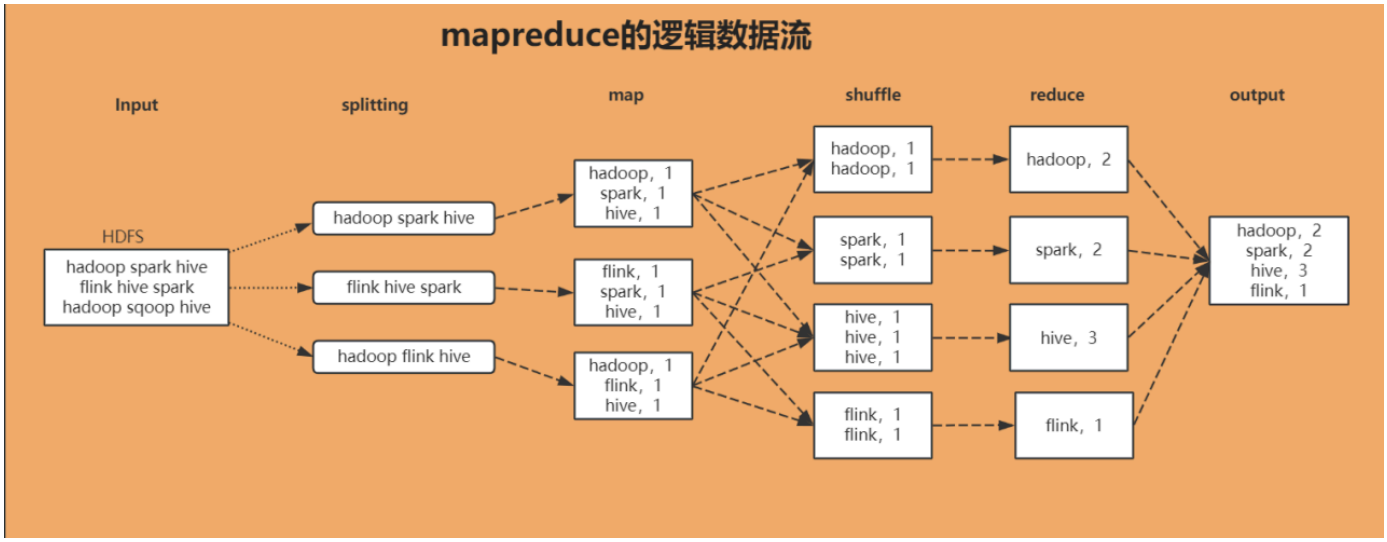
参考答案：

MapReduce的核心思想是化大为小，分而治之。

Mapreduce的主要组成：mapper和reducer

Mapper阶段：负责“分”，讲复杂的任务分解成若干个“简单的任务”来并行处理。Map阶段的任务没有依赖关系，所以可以并行的来处理。

Reducer阶段：负责map阶段的结果的全局汇总。



11.Mapreduce中combiner发生在什么时候，什么时候使用?

课程链接：5.4.5.Combiner函数

参考答案：

MapReduce 中 Combiner 发生的时机及使用场景

`Combiner`（本地聚合器）是 **MapReduce** 中的一个可选优化步骤，它可以在 **Map 阶段后** 对数据进行 **局部聚合**，减少 **网络传输量**，从而提高作业效率。

1. Combiner 发生的时机（什么时候运行？）

Combiner 在 Map 阶段和 Reduce 之间运行

1. **Map 阶段完成后**，每个 Mapper 生成 **本地的中间键值对（key-value）**。
2. **在数据写入磁盘之前，或者在 Map 端分区（Partition）之前，Combiner 可能被调用。**
3. **Combiner 的作用是先对 Mapper 输出的数据进行局部聚合，减少网络传输数据量。**
4. **Reduce 阶段再进行最终的全局聚合。**

2. 什么时候使用 Combiner?

适用场景（适用于可以进行局部聚合的计算）

1. **计数操作**（比如 `word count`）：
2. **求和操作（SUM）：**
3. **最大值、最小值计算（MAX / MIN）**

不适用的场景（不适用于非幂等、不满足结合律的计算）

1. **不能用于排序**
2. **不能用于非幂等操作（如计算均值、方差）**

12.小文件的产生和治理

参考答案：

在 Hadoop（特别是 HDFS）中，**小文件（Small Files）是指 远小于 HDFS 块大小（默认 128MB）的文件**，如 几 KB 或几 MB 的文件。这些小文件会 严重影响 HDFS 和 MapReduce 的性能，因此需要有效治理。

1. 小文件的产生

1.1 小文件的来源

1. **日志文件：** 例如 Web 服务器日志、应用日志、传感器数据，每个日志文件可能只有几 KB 或 MB。
2. 爬虫抓取数据：互联网爬虫获取的大量网页，可能是 成千上万个小 HTML 文件。
3. 数据库导出： 例如 MySQL 导出数据到 HDFS，每天可能会生成 多个小 CSV 文件。
4. 实时流式数据： 例如 Kafka、Flume 采集的数据，每个时间窗口可能生成一个小文件。
5. 业务代码错误： 可能某些任务 分片方式不合理，导致数据分布在多个小文件中。

1.2 为什么小文件会影响 HDFS?

1. **NameNode 负担加重（元数据存储问题）**
   - HDFS 元数据存储在 NameNode 内存，每个文件 需要占用 150-200 字节 内存。
   - 如果 HDFS 有 1 亿个小文件，NameNode 可能需要 20GB 内存！
   - 大多数 Hadoop 集群，NameNode 内存是瓶颈，导致整个 HDFS 性能下降。
2. **MapReduce 任务性能下降**
   - 每个文件会启动一个 Map 任务，如果有 100 万个小文件，就会 启动 100 万个 Map 任务，调度和 IO 开销巨大。
   - 小文件数据量小，但作业启动时间长，任务调度延迟严重。
3. **磁盘、网络 IO 开销增加**
   - 读取小文件时，每次 HDFS 需要打开、读取、关闭文件，比大文件读取慢很多。
   - 小文件存储不连续，读取时会 大量随机 IO，降低磁盘和网络吞吐量。

2. 小文件的治理方案

合并小文件：使用 Hadoop Archive（HAR）；使用 MapReduce 合并小文件；使用 HDFS concat 命令

优化存储格式： 适用于：Hive、Spark SQL、大数据仓库

- 使用 ORC / Parquet
- 使用 SequenceFile 存储小文件

优化数据采集：Kafka、Flume、Sqoop 等数据采集工具

- 调整 Kafka / Flume 写入策略
- Sqoop 导入数据时合并

3. 结论

治理方案	适用场景	优点	缺点
Hadoop Archive（HAR）	HDFS 历史数据	节省 NameNode 元数据	不减少物理小文件
MapReduce 合并小文件	日志数据	彻底合并小文件	需要额外 MapReduce 任务
HDFS concat	HDFS 现有小文件	直接合并物理文件	只能合并同目录文件
ORC / Parquet	数据仓库（Hive）	高压缩、查询快	仅适用于 SQL 场景
SequenceFile	Key-Value 存储	读取效率高	需要特殊解析
优化 Kafka / Flume	实时数据采集	减少小文件产生	可能影响实时性
优化 Sqoop 导入	MySQL → HDFS	避免小文件	影响并行性能

13.MR优化的方式

尖叫提示：写了很多，你随便记两个，然后做下解释，不用全部记

问过的一些公司：25.得物社招数开面试题

参考答案：

MapReduce 是 Hadoop 生态系统中的核心计算框架，但其 默认实现可能存在性能瓶颈，如 任务调度慢、I/O 过多、数据倾斜等。优化 MapReduce 可以 减少计算时间、降低资源消耗、提高吞吐量。

1. 任务调度优化：**增加 Map 任务：减少 Reduce 任务（如果数据量小）**

   调整 Map 和 Reduce 数量：默认情况下，Hadoop 自动分配 Map 和 Reduce 任务，但可能不够优化。

2. 数据本地性优化：提高数据本地性（Data Locality）：调整参数，**让任务尽可能在 数据所在节点执行**：

   默认情况下，Map 任务可能运行在 非数据所在节点，导致 网络传输成本高。

3. I/O 优化

（1）使用压缩减少 I/O：使用 Snappy 压缩（轻量级压缩，性能高）：

（2）使用合适的存储格式（Parquet/ORC）：改用 Parquet/ORC（列式存储）格式，提高查询和分析性能

（3）启用 Combiner：使用 Combiner（本地 Reduce 先聚合数据）

（4）优化 Shuffle 过程：增加内存缓冲区，减少磁盘 I/O；减少 Map 输出到磁盘的次数

4. 任务执行优化

（1）避免数据倾斜（Data Skew）：使用自定义分区器（Partitioner）

（2）增大 Reducer 并行度：增加 Reduce 任务数量

14.MR跑的慢的原因：减少落盘次数，那不落盘的话，数据放在哪里，是内存吗?

衍生问题： spark shuffle与MR的不同

参考答案：

是的，MR（MapReduce）运行慢的主要原因是**磁盘 I/O 过多**，每个阶段（Map -> Shuffle -> Reduce）都需要**将中间数据落盘（写入 HDFS 或本地磁盘）**。

如果减少落盘次数，那数据就需要存放在**内存**，但 MR 主要基于**磁盘计算**，默认不会将数据存放在内存里。相比之下，**Spark 采用 RDD（弹性分布式数据集）存储在内存中**，所以计算速度比 MR 快很多。

Spark 为什么比 MR 快?

1. Spark 采用 RDD（Resilient Distributed Dataset，弹性分布式数据集），默认将中间数据存放在内存，大大减少了磁盘 I/O。
2. 任务失败时，RDD 采用血缘（Lineage）机制进行重算，只需要重新计算丢失的部分，而 MR 需要重新执行整个任务。
3. DAG（有向无环图）优化执行计划，避免了 MR 的多次落盘问题，提高执行效率。

总结

- MR 运行慢的核心原因：频繁落盘，磁盘 I/O 高。
- 减少落盘后，数据存放在内存，但 MR 本身不支持这样做，Spark 通过 RDD 实现了内存计算，比 MR 更快。

15.没有排序需求就可以使用hashshuffle吗?

参考答案：

是的，如果 **没有排序需求**，可以使用 **HashShuffle**，这样可以**减少磁盘 I/O，提高性能**。

1. 什么是 HashShuffle?

HashShuffle（哈希分区 Shuffle） 是一种 **基于哈希分区** 的数据传输方式，它的基本思路是：

- **每个 Map 任务** 计算后，通过 **哈希（Hash）函数** 将数据分配到不同的 Reduce 任务对应的分区。
- **每个 Reduce 任务** 只需要拉取自己对应的分区数据，并进行处理。

这样，数据传输时**不会进行排序**，也不需要复杂的合并（Merge）操作，**性能比排序 Shuffle 高**。

2. 什么时候可以用 HashShuffle?

你可以使用 HashShuffle **的前提条件是：不需要排序**，主要适用于：

- **Group By 统计**（如 `groupByKey()`）—— 只要数据能正确分组，不要求有序。
- **数据聚合（Aggregation）**（如 `reduceByKey()`、`countByKey()`）。
- **连接操作（Join）** —— 只要 Join 逻辑不依赖排序，使用 `hashPartitioner` 即可。

3. 什么时候不能用 HashShuffle?

如果 **数据必须有序**，则不能使用 HashShuffle，例如：

- **排序操作（SortBy、OrderBy）**：需要 `SortShuffle` 或 `RangeShuffle` 进行排序。
- **全局 Top N 操作**：需要数据按顺序分区，使用 `SortShuffle` 更合适。

16.spill过多，怎么优化

参考答案：

在 **MapReduce（MR）** 中，**Spill（溢写）** 发生在 **内存缓冲区不足** 时，数据被写入**本地磁盘**以腾出空间，这会导致磁盘 I/O 过多，降低任务性能。**优化 Spills 主要是减少磁盘写入，提高内存利用率。**

(1) 调整 Map 端缓冲区，

增加内存，减少 Spill 触发：这个参数控制 Map 端用于存储中间数据的 **排序缓冲区大小**，默认值一般是 **100MB**。适当增大（如 **256MB 或 512MB**），减少写磁盘的次数。

调整溢写触发阈值：控制 缓冲区达到多少比例时开始 Spill，默认是 **0.80（80%）**，适当增大（**如 0.9**），让更多数据先存内存，减少 Spill。

(2) 优化 Reduce 端合并，减少溢写

控制 Reduce 拉取数据时的内存：Reduce 任务拉取数据时，存放在内存中的比例，默认是 **0.70（70%）**。适当增大，比如 **0.8**，减少落盘。

减少 Reduce 端 Merge 过程：规定 **Reduce 端最多在内存中合并多少个 Map 任务的数据**，超出后才会写磁盘。适当 **增大**（如 **100**），减少 Merge 过程中的磁盘 I/O。

(3) 减少小文件，优化输入格式：

使用 `CombineTextInputFormat` 合并小文件：如果输入数据是很多小文件，MR 会为每个小文件创建 Map 任务，导致内存浪费，增加 Spill 发生的可能。

(4) 适当增加 Reducer 数量：如果 Reduce 任务的数据量过大，Spill 过多，可以 **适当增加 Reducer**（比如调大到 2 倍）。

17.数据倾斜可能是什么原因，怎么发现的，如何处理

参考答案：

数据倾斜（Data Skew） 发生在 **MapReduce、Spark、Flink** 等大数据计算框架中，主要是因为**部分 Key 对应的数据量过大，导致某些 Task 处理时间远超其他 Task**，从而拖慢整个 Job。常见原因包括：

Key 分布不均匀：某些 Key 的数据远多于其他 Key，导致部分 Reducer 负载过重。

数据倾斜导致某些 Task 处理时间过长：在 MR 或 Spark 的 Shuffle 阶段，某些 Key 对应的数据量特别大，导致某些 Reducer 任务需要处理 **GB 级别数据**，而其他 Reducer 可能只处理 **KB 级别数据**，最终影响整体任务进度。

2. 如何发现数据倾斜

(1) 观察任务执行时间：在 Spark UI 或 Hadoop Job Tracker 中，如果某些 Task 运行时间特别长，可能是数据倾斜导致的；Spark UI -> Stages -> Tasks，查看 Task 处理数据的大小和执行时间。

(2) 检查数据分布

3. 解决方案

(1) **预处理数据，**避免 Key 过于集中

(2) **增加随机前缀，**打散 Key：在 Key 前面添加随机数 打散数据，然后在 Reduce 端再合并：

(3) **扩大 Reduce 任务数量：**调整 Reducer 数量

(4) **使用 `salting` 机制：**在 Key 上添加随机数，打散数据；Join 之后再去掉随机前缀

(5) **使用 `skewed join` 处理数据倾斜：**Spark 2.x+ 版本支持 `skewed join` 机制让 Spark 自动检测数据倾斜，并优化 Join 过程。

18.Shulffle过程瓶颈在哪里，你会怎么解决?

参考答案：

Shuffle 过程瓶颈

在 MapReduce、Spark、Flink 这些大数据框架中，Shuffle 过程是数据从 Map 任务传输到 Reduce 任务的阶段，涉及 数据分区、排序、网络传输、磁盘存储 等多个环节。主要瓶颈包括：

(1) **磁盘 I/O 过高：**Map 任务的中间数据需要落盘（spill），Reduce 任务拉取数据时可能涉及磁盘读取，导致 I/O 瓶颈。

(2) **网络传输压力大：**Reduce 任务需要从多个 Map 任务获取数据，数据量大时网络传输成为瓶颈。

解决方法：压缩 Shuffle 数据：增大 Reduce 端 Buffer，避免频繁请求：

(3) **数据倾斜：**如果某些 Key 对应的数据远超其他 Key，会导致某些 Reduce 任务负载过重，执行时间远超其他任务。

(4) **Shuffle 读写文件太多：**Map 任务会生成多个小文件，Reduce 任务再读取，导致 I/O 瓶颈。

(5) **Reduce 任务过少，导致单个任务负载大：**Reduce 任务数太少，导致部分任务压力大。

郑冬综合2502 2026/9/10 22:03:50
涤生大数据版权所有，请勿外传

无标签